COMPUTER SCIENCE

PAPER 1 (THEORY)

Maximum Marks: 70 **Time Allowed: Three Hours**

Reading Time: Additional Fifteen Minutes

Instructions to Candidates

1. You are allowed an additional fifteen minutes for only reading the question paper.
2. You must NOT start writing during the reading time.
3. This question paper has 12 printed pages.
4. It is divided into two parts: Part I and Part II.
5. It has eleven questions in all.
6. Part I is compulsory and has two questions.
7. While attempting Multiple Choice Questions in Part I, you are required to write only ONE option as the answer.
8. Part II is divided into three sections: A, B and C.
9. Each section in Part II has three questions. Any two questions have to be attempted from each section.
10. The intended marks for questions are given in brackets [ ].

Instruction to Supervising Examiner

1. Kindly read aloud the Instructions given above to all the candidates present in the examination hall.

PART I – 20 MARKS

Answer all questions.

While answering questions in this Part, briefly indicate your working and reasoning, wherever required.

Question 1

(i) Consider the class given below, where show( ) is a static member method: [1]

```
class Student
{
    int roll;
    static int count;
    void greet()
    {
        System.out.println("Hello");
    }
    static void show()
    {
        ...
    }
}
```

Which of the following statements are valid for show( )?

I. It can be invoked as `Student.show( )` without creating any object.

II. It can call the non-static method `greet( )` directly by its name.

III. It cannot refer to `this` or `super`.

(Application)

(a) I and III

(b) II and III

(c) I and II

(d) Only III

(ii) Simplification of the Karnaugh map given by F(A, B, C, D) = Σ(5, 7, 13, 15) gives: [1]
(Application)

(a) A′ • B

(b) A + B • D′

(c) A • D′

(d) B • D

(iii) A Boolean function F(A, B, C) = A′•B′•C′ + A•B′•C + A•B•C′ has its shorthand form as: [1]
(Application)

(a) $m_0 + m_3 + m_6$

(b) $m_0 + m_5 + m_6$

(c) $m_1 + m_5 + m_6$

(d) $m_0 + m_5 + m_7$

(iv) According to the Principle of Duality, the Boolean equation (P′ + Q • 0) • P = 0 will be equivalent to: *(Application)* [1]

(a) (P′ • (Q + 1)) + P = 0

(b) (P′ + (Q • 1)) • P = 1

(c) (P′ • (Q + 1)) + P = 1

(d) (P • (Q + 1)) + Q = 1

(v) The complement of the Boolean expression (P′ • Q) + (Q • R′) is: *(Application)* [1]

(a) (P′ + Q) • (Q + R′)

(b) P • Q′ + Q • R

(c) (P + Q′) • (Q′ + R)

(d) (P + Q) • (Q′ + R′)

(vi) The contrapositive of p ⇒ q′ is: *(Application)* [1]

(a) q′ ⇒ p

(b) q ⇒ p′

(c) p′ ⇒ q

(d) q′ ⇒ p′

(vii) Given below are two statements marked Assertion and Reason. Read the two statements carefully and choose the correct option. [1]

Assertion: A class that implements an interface must implement all of its methods (unless the class is itself declared abstract).
Reason: If a class does not implement all the methods of an interface, it can still be instantiated.

(Analysis)

(a) Both Assertion and Reason are true and Reason is the correct explanation for Assertion.

(b) Both Assertion and Reason are true but Reason is not the correct explanation for Assertion.

(c) Assertion is true and Reason is false.

(d) Both Assertion and Reason are false.

(viii) Given below are two statements marked Assertion and Reason. Read the two statements carefully and choose the correct option. [1]

Assertion: When an array is passed to a method, any change made to its elements inside the method is reflected in the array of the calling method.
Reason: Only the reference to the array is passed, so the formal parameter and the actual parameter refer to the same array object.

(Analysis)

(a) Both Assertion and Reason are true and Reason is the correct explanation for Assertion.

(b) Both Assertion and Reason are true but Reason is not the correct explanation for Assertion.

(c) Assertion is true and Reason is false.

(d) Both Assertion and Reason are false.

(ix) State the Involution law. Verify it using a truth table. *(Understanding)* [1]

(x) What is the space complexity of the recursive factorial function `int fact(int n)`? [1]
(Understanding)

Question 2

(i) Convert the following infix expression into polish notation / prefix form: [2]
M + (N * P − (Q / R ^ S) * T) * U *(Application)*

(ii) Each element of an array Z[−12 … 18][4 … 25] is of type `short` (2 bytes). If the array is stored in column major order beginning at location 1000, determine the location of the element Z[3, 20]. *(Application)* [2]

(iii) The following function tally( ) is a part of some class. Assume that n, x and y are positive integers. Answer the questions given below along with the dry run / working.

```
int tally(int n, int x, int y)
{
    if (n < 10)
        return n;
    else
    {
        int z = n % 10;
        if (z % 2 == 0)
            return z * x + tally(n / 10, x, y);
        else
            return z * y + tally(n / 10, x, y);
    }
}
```

(a) What will be the output of tally(n, x, y) if n = 5273, x = 2 and y = 3? *(Analysis)* [2]

(b) What is the function tally( ) performing, apart from recursion? *(Analysis)* [1]

(iv) The following recursive function of class Conv converts a string of digits into the integer it represents (for example, "1235" becomes 1235). Some statements or expressions are missing and are marked by ?1?, ?2? and ?3?. Replace each with suitable code so that the function works correctly.

```
static int toInt(String str)
{
    if (str.length() == ?1?)
        return str.charAt(0) - ?2?;

    double y = toInt(str.substring(1));
    double x = str.charAt(0) - '0';
    x = x * ?3? + y;
    return (int) x;
}
```

(a) What expression should replace ?1? *(Analysis)* [1]

(b) What expression should replace ?2? *(Analysis)* [1]

(c) What expression should replace ?3? *(Analysis)* [1]

PART II – 50 MARKS

Answer six questions in this part, choosing two questions from Section A, two from Section B and two from Section C.

SECTION A

Answer any two questions.

Question 3

(i) A school library allows a student to borrow a reference book for home use only if any one of the following conditions is satisfied: [5]

- The student is a member of the library and has no dues.

  OR
- The student is not a member but holds a permission slip from the librarian, and has no dues.

  OR
- The student has been nominated by a teacher, but is neither a member nor holds a permission slip.

The inputs are:

INPUTS

M	The student is a member of the library.
P	The student holds a permission slip from the librarian.
N	The student has been nominated by a teacher.
D	The student has dues.

(In all the above cases, 1 indicates yes and 0 indicates no.)
Output: X – denotes that the book can be borrowed [1 indicates yes and 0 indicates no in all cases].

Draw the truth table for the inputs and the output given above and write the SOP expression for X(M, P, N, D).

(Application)

(ii) Reduce the above expression X(M, P, N, D) by using a 4-variable Karnaugh map, showing the various groups (i.e. octal, quads and pairs). Draw the logic gate diagram for the reduced expression using NAND gates only. Assume that the variables and their complements are available as inputs. *(Create)* [5]

Question 4

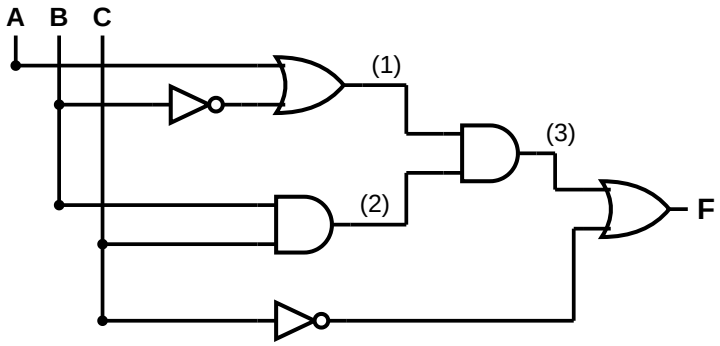
(i)

(a) Given the Boolean function F(A, B, C, D) = Σ(1, 3, 5, 6, 7, 9, 11, 14, 15), reduce the above expression by using a 4-variable Karnaugh map, showing the various groups (i.e. octal, quads and pairs). *(Application)* [4]

(b) Draw the logic gate diagram for the reduced expression using basic gates. Assume that the variables and their complements are available as inputs. *(Create)* [1]

(ii)

(a) From the logic circuit diagram given below, name the outputs (1), (2) and (3) and finally derive the Boolean expression F and simplify it. State whether F is a tautology, a contradiction or a contingency. *(Application)* [4]



(b) Write the dual of the expression (A + 0) • B. *(Application)* [1]

Question 5

(i) Draw the logic circuit to decode the following binary numbers (0001, 0011, 0110, 1000, 1010, 1011, 1100, 1110) to their hexadecimal equivalents. Also state the hexadecimal equivalents of the given binary numbers. *(Create)* [5]

(ii) Verify, using a truth table, whether the following proposition is valid or not: [3]

$((X \wedge Y) \wedge Z) \Rightarrow (X \wedge Y \wedge Z) = (X \wedge Y) \Leftrightarrow (X \wedge Y \wedge Z)$ *(Analysis)*

(iii) Represent the given expression in canonical and cardinal POS form. [2]

$H(x, y, z) = x'•y•z' + x•y'•z' + x•y•z + x'•y•z$ *(Application)*

SECTION B

Answer any two questions.

Each program should be written in such a way that it clearly depicts the logic of the problem. This can be achieved by using mnemonic names and comments in the program.

(Flowcharts and Algorithms are not required.)

The programs must be written in Java.

Question 6

A fraction is a combination of two positive integers, a numerator and a denominator. A fraction is in its **lowest terms** when its numerator and denominator have no common factor except 1. Design a class Fraction to reduce a fraction to its lowest terms, and to add two fractions. [10]

```
Example 1: 35/77 in lowest terms is 5/11
Example 2: 1/6 + 3/10 = 28/60 = 7/15
```

The details of the members of the class are given below:

Class name	:	**Fraction**
Data members/instance variables:		
n, d	:	integer variables to store the numerator and the denominator respectively
Methods/Member functions:		
Fraction(int a, int b)	:	parameterised constructor to initialise the data members
int fnHCF(int a, int b)	:	to return the highest common factor of a and b using the **recursive technique**
Fraction fnReduce(Fraction f1)	:	to reduce the fraction f1 to its lowest terms using fnHCF( ) and to return it as a new object
Fraction fnAdd(Fraction f1)	:	to add the current fraction and f1 and to return the sum in its lowest terms, by calling fnReduce( )
void fnShow( )	:	to display the fraction in the form n/d

Specify the class Fraction giving details of the constructor( ), int fnHCF(int, int), Fraction fnReduce(Fraction), Fraction fnAdd(Fraction) and void fnShow( ). Define a main( ) function to create objects and call the functions accordingly to enable the task.

(Create)

Question 7

A class Pool contains an array of at most 100 integers. Using the following class description, create an array holding the **common elements** of two integer arrays: every value that occurs in both arrays, listed once, in the order in which it occurs in the first array. [10]

```
Example: first array  : 4 7 9 7 12
         second array : 7 12 3 4
         common       : 4 7 12
```

Some of the members of the class are given below:

Class name : **Pool**

Data members/instance variables:

a[ ]	:	an array of 100 integers
len	:	length of the array (number of elements)

Methods/Member functions:

Pool( )	:	default constructor
Pool(int n)	:	to initialise the length of the array, len = n
void readArray( )	:	to input the array elements
Pool common(Pool p)	:	to return the Pool containing the common elements of the current Pool object and the Pool object passed as the parameter
void display( )	:	to display the array elements; if there are none, to display the message “No common elements”

Specify the class Pool giving details of the constructors and the functions given above (you may add a helper method). Define the main( ) function to create objects and call the methods accordingly to enable the task.

(Create)

Question 8

A class PalPrime has been defined to display the **PalPrime** words of a sentence. A PalPrime word is a **palindrome** whose **length is a prime number**. The sentence is in UPPER CASE and its words are separated by a single blank space. [10]

```
Example 1: Input : WOW DID YOU SEE THE REFER AND RADAR TOO
           Output: WOW, DID, REFER, RADAR
                   NUMBER OF PALPRIME WORDS = 4
Example 2: Input : A MADAM SAW LEVEL ABBA
           Output: MADAM, LEVEL
                   NUMBER OF PALPRIME WORDS = 2
```

(In the second example A is a palindrome of length 1, and ABBA one of length 4; neither length is prime.) Some of the members of the class are given below:

Class name	:	**PalPrime**
Data members/instance variables:		
line	:	to store a sentence
Methods/Member functions:		
PalPrime( )	:	constructor to initialise the data member
void accept( )	:	to accept the sentence in UPPER CASE
boolean isPalin(String w)	:	returns true if the word w is a palindrome, otherwise false
boolean isPrime(int n)	:	returns true if n is a prime number, otherwise false
void display( )	:	to extract each word of the sentence and display the PalPrime words, separated by commas, with their count, by invoking isPalin( ) and isPrime( ); if there is none, to display “NO PALPRIME WORDS”

Specify the class PalPrime giving details of the constructor( ), void accept( ), boolean isPalin(String), boolean isPrime(int) and void display( ). Define a main( ) function to create an object and call the functions accordingly to enable the task.

(Create)

SECTION C

Answer any two questions.

Each program should be written in such a way that it clearly depicts the logic of the problem stepwise. This can be achieved by using comments in the program and mnemonic names or pseudo codes for algorithms. The programs must be written in Java and the algorithms must be written in general / standard form, wherever required / specified.

(Flowcharts are not required.)

Question 9

(a) Define a class RingQ which works on the FIFO principle: the user enters data from the rear end and removes data from the front end, and the positions freed at the front are reused, the rear end being connected to the front end in a circular arrangement. The details of the class are given below: [4]

Class name : **RingQ**

Data members/instance variables:

q[ ]	:	to store the integer elements
cap	:	to store the maximum capacity of the array
f	:	to point to the index of the front end
r	:	to point to the index of the rear end

Methods/Member functions:

RingQ(int max)	:	parameterised constructor to initialise cap = max, f = 0 and r = 0 and to create the array
void push(int n)	:	to add an element from the rear end if possible, otherwise display the message “QUEUE FULL”
int pop( )	:	to remove and return an element from the front if there is any, otherwise return −999
void display( )	:	to display the queue elements

Specify the class RingQ giving details of the functions void push(int) and int pop( ). Assume that the other functions have been defined. The main( ) function and algorithm need NOT be written.

(Create)

(b) State one advantage of a circular queue over a linear queue. *(Recall)* [1]

Question 10

A super class Resort has been defined to store the details of a resort. Define a sub class Guest to calculate the total bill of a guest, as per the following criteria: the room rent for the whole stay (rent per day × number of days) is charged, and a room of type **Deluxe** carries an additional amount of 12% of that rent, a room of type **Villa** an additional 25%; a **Standard** room has no additional amount. The details of the members of both the classes are given below: [5]

Class name : **Resort**

Data members/instance variables:

rname : to store the name of the resort

rent : to store the room rent per day (in decimals)

Methods/Member functions:

Resort(…) : parameterised constructor to assign values to the data members

void show( ) : to display the resort details

Class name : **Guest**

Data members/instance variables:

gname : to store the name of the guest

days : to store the number of days of stay

type : to store the room type (“Standard”, “Deluxe” or “Villa”)

bill : to store the total bill (in decimals)

Methods/Member functions:

Guest(…) : parameterised constructor to assign values to the data members of both the classes

void calculate( ) : to calculate the total bill as per the criteria given above

void show( ) : to display the resort details along with the guest details and the bill

Assume that the super class Resort has been defined. Using the concept of inheritance, specify the class Guest giving the details of the constructor(…), void calculate( ) and void show( ). The super class, main function and algorithm need NOT be written.

(Create)

Question 11

(i) A linked list is formed from the objects of the class Node. The class structure of the Node is given below: [2]

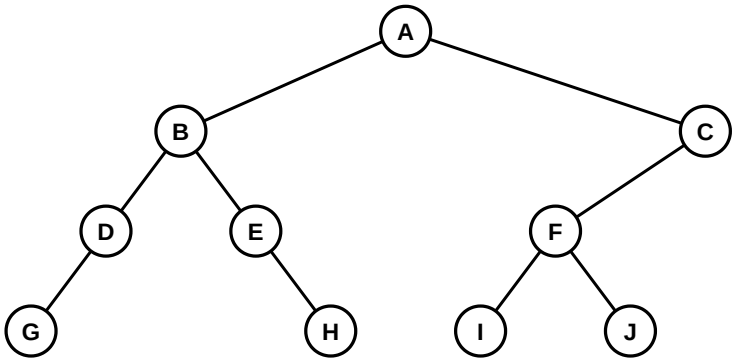
```
class Node
{
    int n;
    Node link;
}
```

Write an Algorithm OR a Method to insert a new element x at the position pos (the first node is at position 1) of an existing linked list and return the reference to the first node of the list. A position one beyond the last node appends the element; if pos is larger than that, or less than 1, the list is left unchanged. The method declaration is as follows:

```
Node insertAt(Node start, int x, int pos)
```

(Analysis)

(ii) Answer the following questions from the diagram of a Binary Tree given below. (The root is at depth 0.)



(a) Write the pre-order traversal of the tree. *(Understanding)* [1]

(b) State the degree of the nodes B and E. *(Understanding)* [1]

(c) State the depth of the node J and the size of the tree. *(Understanding)* [1]

— — — —